

Smart Hospital and Resource Allocation System

Technical Project Report



T. S. P. Fernando - AS20250397

Department of Computer Science
University of Sri Jayewardenepura

Submitted as part of the requirements for the
CSC 1012 Introduction to Computer Programming

Submission Date: 20th September
2026

GitHub Repository:
<https://github.com/SenithuPiumjith/Smart-Hospital-Resource-Allocation-System.git>

High Level Architecture and Function List

This system follows a modular architecture separating user interaction, utility calculations, formatting and file persistence. The functions are structured as follows.

Menu and Program Control

- **main()** : Manages the application lifecycle, loads persistent data on startup, handles the main execution loop, and saves bed state before termination.
- **displayMenu()** : Shows the primary user interface menu option to standard output.
- **resetSystem()** : Additionally add this function to perform a complete system wipe by zeroing out `beds_status.txt` and `patient_records.txt`

Core Workflow Controllers

- **registerPatient()** : Manages the full patient intake lifecycle. Collecting the patient's personal details, emergency level, ward requirements, assigning beds and calculating all costs, printing the receipt and writing the record to permanent log.
- **displaySortedPatients()** : Implements a sorting algorithm (bubble sort) to output all register patient records according to medical priority (critical first).
- **generateReport()** : Output an admin report detailing patient intake by triage level, financial revenue and discounts. Also shows ward occupancy rate and the highest paying patient.
- **displayBedStatus()** : Generate a visual availability for every bed in every ward.

Mathematical Calculations

- **assignBed()** : Scans the relevant ward's row in the array for an available slot (0) and marks it occupied (1) and returns the assigned bed number.
- **calculateWaitTime()** : Compute total wait time in minutes based on current queue length and doctor consultation duration.
- **calculateSurcharge()** : Applies emergency surcharges based on triage level (0% for Level 1, 20% for Level 2, 50% for Level 3).
- **calculateWardCost()** : Calculates total ward stay charges based on daily rates and admitted days.
- **calculateGrossTotal()** : Sums base consultation fee, emergency surcharge, and total ward stay cost.
- **calculateDiscount()** : Evaluates age criteria to apply a 15% discount on gross total for patients under 5 or over 65 years.
- **calculateFinalAmount()** : Subtracts total discount from gross total bill to yield payable amount

Output and Formatting

- **formatCurrency()** : Formats floating point amounts into strings with comma separators for thousands.
- **printBill()** : Prints a detailed admission statement and bill receipt for patient record.

Data Persistence

- **saveBedStatus()** : Writes the current 2D bed occupancy matrix to beds_status.txt.
- **loadBedStatus()** : Reads saved matrix states from beds_status.txt on program initialization.
- **savePatientRecord()** : Appends register patient billing transactions to patient_records.txt.

Data Structure Design

While the project specification permitted parallel arrays, **struct** types were implemented to group related attributes into cohesive units.

```
1 struct doctorSpecRow {
2     int specialtyId;
3     const char* specialtyName;
4     double baseFee;
5     int time;
6     int dailyCap;
7 };
8 struct wardsDataRow {
9     int wardId;
10    const char* wardName;
11    double bedRate;
12    int bedCap;
13 };
14 struct patientRecords {
15     char name[50];
16     int age;
17     int triageLevel;
18     int specId;
```

```
19   int admittedFlag;
20   int wardId;
21   int daysAdmitted;
22   int bedNum;
23   double waitTime;
24   double baseFee;
25   double surcharge;
26   double wardCost;
27   double grossTotal;
28   double discount;
29   double finalAmount;
30 };
```

Listing 1: Core Data Structure Definitions for Specialities, Wards and Patient Records.

Structs eliminate the risk of index desynchronization across multiple arrays, make function cleaner and improve maintainability. Also 2D array matrix used to store bed availability states (0 = Available, 1 = Occupied) where rows represent ward IDs and columns represent bed slot numbers.

System Assumptions

- 1. Ward Full Fallback :** If an attempted ward admission fails due to maximum capacity, the patient is reassigned to Outpatient (OPD) status (`admittedFlag = 0`, `wardID = 0`, `daysAdmitted = 0`) instead of rejecting registration.
- 2. Immediate Attention Display :** The string (Immediate Attention) is triggered when computed waiting time evaluates to 0.00 minutes.
- 3. Array Boundaries :** Total system capacity is statically capped at `MAX_PATIENTS = 100`.
- 4. Bed Number Offset :** Beds are stored internally using 0 based array indices (0 to `bedCap - 1`) but displayed in output views as 1 based labels (1 to `bedCap`) for clarity.
- 5. Patient ID Sequence :** Patient's IDs are computed for display as `1000 + index + 1` (eg: PAT - 1001).

6. **Report Revenue Definition** : Total Revenue calculated by net collected income (finalAmount) rather than pre discount gross billing.
7. **Queue Scope** : Specialty queue counts reset on each program execution, as real time waiting queues reflect active operational sessions.
8. **File Handling Scope** : patient_records.txt function strictly as a write only append log ("a") for audit history, whereas beds_status.txt maintains full read and write state.